Refinery Production Planning Optimizer

Code walkthrough – what every statement does · Revision 2

Covers the interactive Streamlit flow app: data_loader.py, model.py, optimizer.py, app.py, run_optimizer.py and the canvas flow_component/index.html. Prepared with Microsoft Copilot, October 2026. All plant data in the app are illustrative sample values.

Revision	Change
Rev. 1	First walkthrough of the flow app (edit → Apply & re-optimize solved immediately).
Rev. 2	Input editor button renamed to **Apply**: it only saves and validates the table. The LP is solved only by **▶ Run optimizer**, by Run settings → Apply & solve, or on first load. Outputs show the last run and are flagged **Re-run needed** until you run. Changes are in app.py and index.html; the other files are unchanged.

Contents

1. How the app fits together 2
2. data_loader.py – Input layer 4
3. model.py – Process model 11
4. optimizer.py – LP optimizer 16
5. app.py – Web host 21
6. run_optimizer.py – Command line 30
7. flow_component/index.html – interactive canvas 32
8. Appendix – input files and statement coverage 35

1. How the app fits together

The app is a pipeline. Each Python file does one job and hands its result to the next. The browser canvas only draws and collects edits; every calculation happens in Python.

Step	File	Input	Output
1 Read & check	data_loader.py	8 CSVs from GitHub (or local data/)	PlanData + list of errors/warnings
2 Model	model.py	PlanData, decision matrix V	Coefficient vectors; daily plan, tank balance, checks, KPIs
3 Optimise	optimizer.py	PlanData + coefficient vectors	Optimised V, binding constraints, shadow prices, explanation
4 Host	app.py	Results of steps 1–3	JSON payload to the canvas; decides when to solve; processes actions sent back
5 Show & edit	flow_component/index.html	Payload	Blocks, flowing connections, pop-up editors and charts; user actions
Alt. CLI	run_optimizer.py	Same CSVs	CSV result files (no browser)

What happens when you edit a value (revised)

A. Apply – save the edit (no solve)

1. You open an input block, change a cell and press **Apply**.
2. index.html sends {nonce, action:'edit', table, columns, rows} to Python (act → setValue).
3. app.py sees a new nonce, rebuilds that table with rows_to_frame() and restarts the script (st.rerun). No solve is requested.
4. data_loader.validate() checks all tables straight away. Errors turn the affected input block red and stop here.
5. If the inputs are valid but differ from the last solved inputs, app.py keeps the last results bundle and sets stale = True.
6. The canvas shows **Re-run needed**: optimizer pill RE-RUN NEEDED, pulsing Run buttons, dashed output blocks, outputs' connections stop flowing, output pop-ups show a 'last-run results' banner.

B. ▶ Run optimizer – solve

7. You press ▶ Run optimizer (toolbar, optimizer block, or the banner in an output pop-up). index.html sends {action:'run'}.
8. app.py sets run_requested = True and restarts the script.
9. data_loader.build() builds PlanData; optimizer.optimize() solves the LP; make_bundle() evaluates the current and optimised plans and packs all output data.
10. The bundle is stored in the cache under the input fingerprint and becomes last_bundle; stale = False.
11. index.html receives the payload with ack = its nonce, clears 'SOLVING…' and redraws everything with the new results.

C. Reset / undo – if the restored inputs were solved before, their cached results are shown instantly without a run.

Key modelling idea

Every quantity (products, costs, specs, tank flows) is a straight-line function of the six daily decisions x_{AL}, x_{MUR}, x_{TAP}, x_{KIM}, r, f. model.coeffs() stores each one as a vector of six numbers, so any quantity on any day is a single multiplication: decisions · vector. The optimizer uses exactly the same vectors to build its constraints,

so the plan it finds and the checks shown afterwards can never disagree. With the sample data the margin per barrel is about 4.62 $/bbl for Arab Light, 1.33 Murban, 0.45 Tapis, 2.00 Kimanis, 11.46 for naphtha → reformer and 8.85 for residue → RFCC.

How to read the file sections

Each Python file is shown in order. Left: line numbers and statement(s); right: what they do. Closely related statements share one explanation. Long lines wrap with ↪. Statements changed in rev. 2 are tagged [NEW] or [REVISED] and shaded orange. index.html is described by function (section 7).

2. data_loader.py – Input layer

Reads the 8 CSVs (GitHub first, local data/ as fallback), checks them, and builds one PlanData object. Unchanged in rev. 2.

Line · statement(s)	What it does

1–5
`"""Load and validate the refinery planning CSVs (from GitHub or the local data/ folder).`
 	
`Pipeline layer 1-2: CSV files -> validated PlanData object.`
`Nothing in here knows about the optimizer or the UI.`
`"""`
	Module description. This file is layers 1–2 of the pipeline: it reads the CSV inputs, checks them and packs them into one PlanData object. It has no optimizer or UI code.

6 `from __future__ import annotations`	Lets the file use modern type hints such as str | Path on older Python versions.

8–11
`import io`
`import urllib.request`
`from dataclasses import dataclass, field`
`from pathlib import Path`
	Standard-library tools: io (treat downloaded text as a file), urllib.request (download from GitHub), dataclass (simple data container), Path (file paths).

13–14
`import numpy as np`
`import pandas as pd`
	numpy for arrays and maths; pandas for tables (DataFrames).

16 `DATA_DIR = Path(__file__).parent / "data"`	The bundled data/ folder next to this file. Used as the fallback when GitHub cannot be reached.

18–27
`FILES = {`
`"crude_data": "crude_data.csv",`
`"unit_data": "unit_data.csv",`
`"product_data": "product_data.csv",`
`"tank_master": "tank_master.csv",`
`"crude_receipts": "crude_receipts.csv",`
`"product_liftings": "product_liftings.csv",`
`"daily_decisions": "daily_decisions.csv",`
`"config": "config.csv",`
`}`
	Maps each internal table name to its CSV file name. Every other part of the code uses these names, so a file is only named in one place.

29–40
`REQUIRED_COLS = {`
`"crude_data": ["crude_id", "crude_name", "tank_id", "price_usd_bbl", "y_lpg", "y_naph", "y_kero",`
`"y_go", "y_ar", "y_loss", "ar_sulfur_wt", "x_min_kbd", "x_max_kbd"],`
`"unit_data": ["param_key", "value"],`
`"product_data": ["product_id", "product_name", "tank_id", "price_usd_bbl"],`
`"tank_master": ["tank_id", "service", "material_id", "type", "capacity_kbbl", "min_heel_kbbl",`
`"opening_kbbl", "end_target_kbbl"],`
`"crude_receipts": ["day", "crude_id", "qty_kbbl"],`
`"product_liftings": ["day", "product_id", "qty_kbbl"],`
`"daily_decisions": ["day", "date", "status"],`
`"config": ["key", "value"],`
`}`
	The columns each CSV must contain. validate() uses this list to catch missing or misspelt columns before any calculation runs.

42–48
`REQUIRED_PARAMS = [`
`"CDU_MAX", "CDU_MIN", "CDU_RAMP", "CDU_OPEX",`
`"REF_MAX", "REF_MIN", "REF_OPEX", "REF_Y_REFORMATE", "REF_Y_LPG",`
`"RFCC_MAX", "RFCC_MIN", "RFCC_OPEX", "RFCC_Y_GASOLINE", "RFCC_Y_LPG", "RFCC_Y_LCO", "RFCC_Y_SLURRY",`
`"DHT_MAX", "DHT_MIN", "DHT_OPEX", "DHT_Y_DIESEL",`
`"RON_REFORMATE", "RON_FCC_GASOLINE", "RON_SPEC_MIN", "AR_SULFUR_MAX",`
`]`
	The param_key values that must exist in unit_data.csv (capacities, opex, yields, RON and sulfur specs). The model cannot run without them.

50 `KNOWN_PRODUCTS = ["LPG", "NAP", "GAS95", "JET", "DSL", "FO"]  # products the process model can make`	Product IDs that model.py knows how to make. A new product needs a new make-equation in model.py, so unknown IDs are rejected.

Line · statement(s)	What it does
51 `YIELD_COLS = ["y_lpg", "y_naph", "y_kero", "y_go", "y_ar", "y_loss"]`	The six CDU cut-yield columns. For each crude they must add up to 1.000 (mass balance).
55 `def read_csvs(folder: str | Path = DATA_DIR) -> dict[str, pd.DataFrame]:`	Reads all eight CSVs from a local folder.
56 `    folder = Path(folder)`	Accepts the folder as text or Path and turns it into a Path object.
57 `    return {name: pd.read_csv(folder / fname) for name, fname in FILES.items()}`	Returns a dictionary {table name: DataFrame}, one entry per file in FILES.
60 `def to_raw_url(url: str) -> str:` 61 `    """Accept a github.com/.../tree/<branch>/<path> link or a raw URL; return the raw base URL."""`	Converts a normal GitHub page link into a raw-file link that returns the actual CSV text.
62 `    url = url.strip().rstrip("/")`	Removes spaces and a trailing "/" so the URL can be joined with file names safely.
63 `    for marker in ("/tree/", "/blob/"):`	GitHub folder links contain /tree/, single-file links contain /blob/. Both forms are handled.
64 `        if "github.com/" in url and marker in url and "raw.githubusercontent.com" not in url:`	Only converts genuine github.com page links that are not already raw links.
65 `            owner_repo, rest = url.split("github.com/", 1)[1].split(marker, 1)`	Splits the link into "user/repo" and "branch/path", e.g. iqbalshafiq96/Self-Work and main/Refinery_Optimizer.
66 `            return f"https://raw.githubusercontent.com/{owner_repo}/{rest}"`	Rebuilds the address on raw.githubusercontent.com, which serves the file contents directly.
67 `    return url`	If no conversion was needed (already raw, or another web address) the URL is returned unchanged.
70 `def read_csvs_from_url(base_url: str, fallback: str | Path = DATA_DIR, timeout: float = 10.0):` 71 `    """Read every CSV from a GitHub folder (tries <base>/<file>, then <base>/data/<file>).` 72 73 `    Any file that cannot be fetched falls back to the local copy.` 74 `    Returns (frames, sources) where sources[name] = URL used, or 'local'.` 75 `    """`	Reads every CSV from a GitHub folder, with a per-file fallback to the local copy. Returns the tables and where each one came from.
76 `    base = to_raw_url(base_url)`	Converts the folder link once into its raw form.
77 `    frames, sources = {}, {}`	Two empty dictionaries: the loaded tables, and the source (URL or "local") of each.
78 `    for name, fname in FILES.items():`	Loops over the eight expected files.
79 `        for url in (f"{base}/{fname}", f"{base}/data/{fname}"):`	Tries two locations in order: directly in the folder, then inside a data/ subfolder.
80 `            try:`	Starts a protected block; a failed download must not crash the app.
81 `                with urllib.request.urlopen(url, timeout=timeout) as r:`	Opens the web address with a time limit (default 10 s).
82 `                    frames[name] = pd.read_csv(io.StringIO(r.read().decode("utf-8-sig")))`	Decodes the downloaded bytes ("utf-8-sig" also strips the invisible marker Excel adds to CSVs) and parses them into a DataFrame.

Line · statement(s)	What it does
83 `sources[name] = url`	Remembers which URL was used, so the sidebar can show "GitHub".
84 `break`	Stops trying the second location once a file has loaded.
85–86 `except Exception: continue`	Any failure (file not found, no internet, bad text) is ignored and the next location is tried.
87 `if name not in frames:`	Neither GitHub location worked for this file.
88 `frames[name] = pd.read_csv(Path(fallback) / fname)`	Loads the bundled local copy instead…
89 `sources[name] = "local"`	…and records that it came from the local folder.
90 `return frames, sources`	Returns both dictionaries.
94–95 `@dataclass class PlanData:`	Declares PlanData: one object holding all cleaned inputs. @dataclass writes the constructor automatically.
96 `crude: pd.DataFrame # index = crude_id`	Crude table (price, yields, sulfur, charge limits), indexed by crude_id.
97 `P: dict # unit parameters, P['CDU_MAX'] ...`	Unit parameters as a lookup, e.g. P["CDU_MAX"] = 150.
98–99 `product: pd.DataFrame # index = product_id` `tank: pd.DataFrame # index = tank_id`	Product table indexed by product_id, and tank table indexed by tank_id.
100–101 `R: pd.DataFrame # receipts (day 1..T x crude_id)` `L: pd.DataFrame # liftings (day 1..T x product_id)`	Receipts as a day × crude matrix and liftings as a day × product matrix (kbbl per day, zeros where nothing happens).
102–104 `decisions: pd.DataFrame # daily_decisions` `cfg: dict` `first_plan_day: int`	The daily decisions table, the config settings, and the first day that the optimizer may change.
105–106 `crudes: list = field(default_factory=list)` `products: list = field(default_factory=list)`	Ordered lists of crude and product IDs. The order fixes the column order of every vector.
108–110 `@property def T(self) -> int: return len(self.decisions)`	T = number of days in the horizon (rows in daily_decisions).
112–114 `@property def var_names(self) -> list[str]: return [f"x_{c}" for c in self.crudes] + ["r_reformer", "f_rfcc"]`	var_names = names of the decision variables for one day: x_AL, x_MUR, x_TAP, x_KIM, r_reformer, f_rfcc.
116–118 `@property def V(self) -> np.ndarray: return self.decisions[self.var_names].to_numpy(float)`	V = the decision values as a T × 6 number matrix. The model works with this matrix.
121–125 `def _num(df, cols): out = df.copy() for c in cols: out[c] = pd.to_numeric(out[c], errors="coerce") return out`	Helper: copies a table and converts the given columns to numbers. Anything that is not a number becomes blank (NaN) so validation can report it.
129–130 `def validate(frames: dict[str, pd.DataFrame]) -> list[tuple[str, str]]: """Return [(level, message)]; level 'error' blocks the run, 'warning' does not."""`	Checks every input table and returns a list of (level, message). "error" blocks the run, "warning" does not.
131 `issues: list[tuple[str, str]] = []`	Empty list that collects the findings.

Line · statement(s)	What it does
132 `err = lambda m: issues.append(("error", m))` 133 `warn = lambda m: issues.append(("warning", m))`	Two short helpers: err() adds an error, warn() adds a warning.
135 `for name, cols in REQUIRED_COLS.items():`	Structural check of every file.
136 `if name not in frames:` 137 `err(f"{FILES[name]} is missing.")` 138 `continue`	File missing → error, skip to the next file.
139 `missing = [c for c in cols if c not in frames[name].columns]`	Lists required columns that are absent from the file.
140 `if missing:` 141 `err(f"{FILES[name]}: missing columns {missing}.")`	Missing columns → error naming them.
142 `if any(lvl == "error" for lvl, _ in issues):` 143 `return issues`	Stops here if structure is broken, because the value checks below would crash on missing columns.
145 `crude = _num(frames["crude_data"], REQUIRED_COLS["crude_data"][3:])` 146 `tank = _num(frames["tank_master"], REQUIRED_COLS["tank_master"][4:])`	Numeric copies of the crude and tank tables for the value checks.
147 `unit, prod, dec = frames["unit_data"], frames["product_data"], frames["daily_decisions"]`	Short names for the unit, product and daily-decision tables.
148 `crudes = crude["crude_id"].astype(str).tolist()` 149 `products = prod["product_id"].astype(str).tolist()`	Lists of crude IDs and product IDs used by later checks.
151 `if crude[REQUIRED_COLS["crude_data"][3:]].isna().any().any():` 152 `err("Crude data: blank or non-numeric values.")`	Any blank or text value in a numeric crude column → error.
153 `for _, r in crude.iterrows():`	Row-by-row crude checks.
154 `s = r[YIELD_COLS].sum()` 155 `if abs(s - 1) > 1e-6:` 156 `err(f"Crude data: yields for {r.crude_id} sum to {s:.4f}, must be 1.000.")`	Adds up the six cut yields; if the total differs from 1.000 → error (mass would be created or lost).
157 `if r.x_min_kbd > r.x_max_kbd:` 158 `err(f"Crude data: {r.crude_id} x_min_kbd > x_max_kbd.")`	Minimum charge rate larger than maximum → error.
159 `if r.price_usd_bbl < 0:` 160 `err(f"Crude data: {r.crude_id} negative price.")`	Negative crude price → error.
161 `if crude["crude_id"].duplicated().any():` 162 `err("Crude data: duplicate crude_id.")`	The same crude ID used twice → error.
164 `keys = set(unit["param_key"].astype(str))` 165 `miss = [k for k in REQUIRED_PARAMS if k not in keys]` 166 `if miss:` 167 `err(f"Unit data: missing param_key {miss}.")`	Collects the parameter keys present and reports any required key that is missing.
168 `vals = pd.to_numeric(unit["value"], errors="coerce")` 169 `if vals.isna().any():` 170 `err("Unit data: blank or non-numeric value.")`	Converts parameter values to numbers; a blank or text value → error.
171 `P = dict(zip(unit["param_key"], vals))`	Temporary parameter lookup for the next checks.
172 `for u in ["CDU", "REF", "RFCC", "DHT"]:` 173 `if P.get(f"{u}_MIN", 0) > P.get(f"{u}_MAX", np.inf):` 174 `err(f"Unit data: {u}_MIN > {u}_MAX.")`	For each unit, MIN must not exceed MAX.
175 `if P.get("REF_Y_REFORMATE", 0) + P.get("REF_Y_LPG", 0) > 1 + 1e-9:` 176 `err("Unit data: Reformer yields exceed 1.0.")`	Reformer reformate + LPG yields may not exceed 1.0.

Line · statement(s)	What it does
177 `if sum(P.get(k, 0) for k in ["RFCC_Y_GASOLINE", "RFCC_Y_LPG", "RFCC_Y_LCO", "RFCC_Y_SLURRY"]) > 1 + 1e-9:` 178 `err("Unit data: RFCC yields exceed 1.0.")`	RFCC gasoline + LPG + LCO + slurry yields may not exceed 1.0.
180 `unknown = [p for p in products if p not in KNOWN_PRODUCTS]` 181 `if unknown:` 182 `err(f"Product data: product_id {unknown} not in the process model {KNOWN_PRODUCTS}.")`	Product IDs the process model cannot make → error.
183 `if pd.to_numeric(prod["price_usd_bbl"], errors="coerce").isna().any():` 184 `err("Product data: blank or non-numeric price.")`	Blank or text product price → error.
186 `if tank[REQUIRED_COLS["tank_master"][4:]].isna().any().any():` 187 `err("Tank master: blank or non-numeric values.")` 188 `else:`	Blank tank numbers → error; otherwise run the per-tank checks.
189 `for _, r in tank.iterrows():` 190 `if r.min_heel_kbbl > r.capacity_kbbl:` 191 `err(f"Tank master: {r.tank_id} min heel > capacity.")` 192 `if not (r.min_heel_kbbl <= r.opening_kbbl <= r.capacity_kbbl):` 193 `warn(f"Tank master: {r.tank_id} opening {r.opening_kbbl:g} is outside [min heel, capacity].")` 194 `if r.end_target_kbbl > r.capacity_kbbl:` 195 `err(f"Tank master: {r.tank_id} end target > capacity.")`	Per tank: min heel above capacity → error; opening stock outside [min heel, capacity] → warning; end target above capacity → error.
196 `for m in crudes + products:` 197 `if (tank["material_id"] == m).sum() != 1:` 198 `err(f"Tank master: material {m} must map to exactly one tank.")`	Every crude and product must be stored in exactly one tank, otherwise the tank balance is ambiguous.
200 `T = len(dec)` 201 `if list(pd.to_numeric(dec["day"], errors="coerce")) != list(range(1, T + 1)):` 202 `err("Actuals: 'day' must run 1..T with no gaps.")`	Horizon length; the day column must run 1, 2, … T without gaps.
203 `need = [f"x_{c}" for c in crudes] + ["r_reformer", "f_rfcc"]`	Expected decision columns for the current crude list.
204 `miss = [c for c in need if c not in dec.columns]` 205 `if miss:` 206 `err(f"Actuals: missing decision columns {miss}.")` 207 `elif _num(dec, need)[need].isna().any().any():` 208 `err("Actuals: blank decision values (use 0 for Plan rows if unknown).")`	Missing decision columns → error; blank decision values → error.
210 `for name, label, idcol, valid in [("crude_receipts", "Crude receipts", "crude_id", crudes),` 211 `("product_liftings", "Product liftings", "product_id", products)]:`	Same set of checks for crude receipts and product liftings.
212 `df = frames[name]`	The table being checked.
213 `bad = sorted(set(df[idcol].astype(str)) - set(valid))` 214 `if bad:` 215 `err(f"{label}: unknown {idcol} {bad}.")`	IDs that do not exist in crude_data / product_data → error.
216 `d = pd.to_numeric(df["day"], errors="coerce")` 217 `if d.isna().any() or ((d < 1) | (d > T)).any() or (d % 1 != 0).any():` 218 `err(f"{label}: every day must be a whole number between 1 and {T}.")`	Each day must be a whole number between 1 and T.
219 `q = pd.to_numeric(df["qty_kbbl"], errors="coerce")` 220 `if q.isna().any() or (q < 0).any():` 221 `err(f"{label}: qty_kbbl must be a number ≥ 0.")`	Each quantity must be a number ≥ 0.
222 `return issues`	Returns all findings to the caller.
226 `def build(frames: dict[str, pd.DataFrame], first_plan_day: int | None = None) -> PlanData:` 227 `"""Turn validated frames into a PlanData object. Call validate() first."""`	Turns validated tables into one PlanData object ready for calculation.
228 `crude = _num(frames["crude_data"], REQUIRED_COLS["crude_data"][3:]).set_index("crude_id")`	Numeric crude table indexed by crude_id.

Line · statement(s)	What it does
229 `unit = frames["unit_data"]`	The unit parameter table…
230 `P = dict(zip(unit["param_key"], pd.to_numeric(unit["value"])))`	…turned into the lookup P[param_key] → number.

231–233 `product = frames["product_data"].copy()`
`product["price_usd_bbl"] = pd.to_numeric(product["price_usd_bbl"])`
`product = product.set_index("product_id")`
	Copies the product table, makes prices numeric and indexes it by product_id.

234 `tank = _num(frames["tank_master"], REQUIRED_COLS["tank_master"][4:]).set_index("tank_id")`	Numeric tank table indexed by tank_id.
235 `dec = frames["daily_decisions"].copy().reset_index(drop=True)`	Copy of the decisions table with a clean 0…T-1 row index.
236 `crudes, products = list(crude.index), list(product.index)`	Ordered crude and product IDs.

237–239 `var = [f"x_{c}" for c in crudes] + ["r_reformer", "f_rfcc"]`
`dec[var] = dec[var].apply(pd.to_numeric)`
`dec["day"] = pd.to_numeric(dec["day"]).astype(int)`
	Decision column names; the next lines force them, and the day column, to be numeric.

240–241 `T = len(dec)` `days = range(1, T + 1)`	Horizon length and the list of days 1…T.
243 `def pivot(df, idcol, cols):`	Inner helper: turns an event list (one row per cargo) into a day × material matrix.
244 `df = df.assign(day=pd.to_numeric(df["day"]).astype(int), qty_kbbl=pd.to_numeric(df["qty_kbbl"]))`	Forces day to whole numbers and quantity to numbers.
245–246 `if df.empty:` `return pd.DataFrame(0.0, index=days, columns=cols)`	No events at all → a matrix of zeros.
247–248 `return (df.pivot_table(index="day", columns=idcol, values="qty_kbbl", aggfunc="sum")` `.reindex(index=days, columns=cols).fillna(0.0))`	Sums quantities per day and material, adds rows/columns for every day and material, and fills gaps with 0.
250–251 `R = pivot(frames["crude_receipts"], "crude_id", crudes)` `L = pivot(frames["product_liftings"], "product_id", products)`	Receipts matrix R (day × crude) and liftings matrix L (day × product).
252 `cfg = dict(zip(frames["config"]["key"], frames["config"]["value"]))`	Config table as a key → value lookup.

254–256 `if first_plan_day is None:`
`plan_rows = dec.index[dec["status"] == "Plan"]`
`first_plan_day = int(dec.loc[plan_rows[0], "day"]) if len(plan_rows) else T + 1`
	No first plan day given → use the first row whose status is "Plan" (or T+1 if all rows are actual).

257 `first_plan_day = int(min(max(first_plan_day, 1), T + 1))`	Clamps the value to the range 1…T+1.
258 `dec["status"] = np.where(dec["day"] < first_plan_day, "Actual", "Plan")`	Relabels every row: days before the first plan day are "Actual" (locked), the rest are "Plan". This is the rolling horizon.
259–260 `return PlanData(crude=crude, P=P, product=product, tank=tank, R=R, L=L, decisions=dec,` `cfg=cfg, first_plan_day=first_plan_day, crudes=crudes, products=products)`	Packs everything into the PlanData object.
263–264 `def load(folder: str | Path = DATA_DIR, first_plan_day: int | None = None) -> PlanData:` `"""Read + validate + build from a local folder. Raises ValueError on input errors."""`	One-call loader for scripts: read local CSVs, validate, build.
265 `frames = read_csvs(folder)`	Reads the files.
266 `errors = [m for lvl, m in validate(frames) if lvl == "error"]`	Keeps only the error messages.
267–268 `if errors:` `raise ValueError("Input data errors:\n- " + "\n- ".join(errors))`	Any error → stop with a readable list of problems.

Line · statement(s)	What it does
269 `return build(frames, first_plan_day)`	Otherwise returns the ready PlanData.

3. model.py – Process model

Turns decisions into unit flows, products, quality, economics, tank balances and constraint checks (the Excel sheets in Python). Unchanged in rev. 2.

Line · statement(s)	What it does

1–7 `"""Process model: decisions (x, r, f) -> flows, products, economics, tank balances and checks.`
 	
`Python twin of the Daily_Plan, Tank_Balance and Constraint_Check sheets.`
`Every quantity is LINEAR in the decision vector v = [x_c1..x_cn, r, f], so each one is stored`
`as a coefficient vector: quantity_t = V[t] @ M['NAME']. The optimizer re-uses the same vectors,`
`so plan evaluation and optimization can never disagree.`
`"""`
	Module description. Key idea: every product, cost and constraint is a straight-line (linear) function of the day's decisions, so each one is stored once as a row of coefficients.

8 `from __future__ import annotations`	Modern type hints on older Python.
10–11 `import numpy as np` `import pandas as pd`	numpy for vector maths, pandas for result tables.
13 `from data_loader import PlanData`	The data container built by data_loader.py.
15 `TOL = 1e-6`	Small tolerance. Checks allow ±0.000001 so rounding noise is not reported as a breach.
18 `def coeffs(d: PlanData) -> dict[str, np.ndarray]:`	Builds the coefficient vector of every linear quantity. Vector order is [x_AL, x_MUR, x_TAP, x_KIM, r, f]; quantity on day t = decisions_t · vector.
19 `P, y = d.P, d.crude`	Short names: P = unit parameters, y = crude table.
20 `nc = len(d.crudes)`	Number of crudes (4).
21 `z = np.zeros(nc)`	Four zeros, used when a quantity does not depend on crude rates.
23–24 `def v(xc, r=0.0, f=0.0):` `return np.r_[xc, r, f].astype(float)`	Helper that assembles one vector: crude coefficients, then the reformer coefficient, then the RFCC coefficient.
26 `M = {`	Dictionary of all coefficient vectors.
27 `"SR_LPG": v(y["y_lpg"].to_numpy(float)),`	Straight-run LPG = Σ (LPG yield of crude c × x_c).
28 `"SR_NAPH": v(y["y_naph"].to_numpy(float)),`	Straight-run naphtha from the CDU.
29 `"SR_KERO": v(y["y_kero"].to_numpy(float)),`	Straight-run kerosene (becomes Jet A-1).
30 `"SR_GO": v(y["y_go"].to_numpy(float)),`	Straight-run gasoil (DHT feed).
31 `"AR": v(y["y_ar"].to_numpy(float)),`	Atmospheric residue (RFCC feed or fuel oil).
32 `"CDU": v(np.ones(nc)),`	CDU throughput = sum of all crude rates.
33 `"REF_FEED": v(z, 1.0),`	Reformer feed = r.
34 `"RFCC_FEED": v(z, 0.0, 1.0),`	RFCC feed = f.
35 `"REFORMATE": v(z, P["REF_Y_REFORMATE"]),`	Reformate = reformer yield × r.

Line · statement(s)	What it does
36 `"REF_LPG": v(z, P["REF_Y_LPG"]),`	Reformer LPG = yield × r.
37 `"FCC_GAS": v(z, 0.0, P["RFCC_Y_GASOLINE"]),`	FCC gasoline = RFCC gasoline yield × f.
38 `"FCC_LPG": v(z, 0.0, P["RFCC_Y_LPG"]),`	FCC LPG = yield × f.
39 `"LCO": v(z, 0.0, P["RFCC_Y_LCO"]),`	Light cycle oil = yield × f (goes to the DHT).
40 `"SLURRY": v(z, 0.0, P["RFCC_Y_SLURRY"]),`	Slurry = yield × f (goes to fuel oil).
41 `"AR_S_MASS": v(y["y_ar"].to_numpy(float) * y["ar_sulfur_wt"].to_numpy(float)),`	Residue sulfur term Σ (residue yield × residue sulfur × x_c), the numerator of the pool sulfur.
42–43 `"CRUDE_COST": v(y["price_usd_bbl"].to_numpy(float)),` `}`	Crude cost per barrel of each crude ($/bbl).
44 `M["DHT"] = M["SR_GO"] + M["LCO"]`	DHT feed = straight-run gasoil + LCO.
45 `make = {`	Product make equations (same as Daily_Plan columns X:AC):
46 `"LPG": M["SR_LPG"] + M["REF_LPG"] + M["FCC_LPG"],`	LPG = SR LPG + reformer LPG + FCC LPG.
47 `"NAP": M["SR_NAPH"] - M["REF_FEED"],`	Naphtha sold = SR naphtha − reformer feed.
48 `"GAS95": M["REFORMATE"] + M["FCC_GAS"],`	Gasoline 95 = reformate + FCC gasoline.
49 `"JET": M["SR_KERO"],`	Jet A-1 = SR kerosene.
50 `"DSL": P["DHT_Y_DIESEL"] * M["DHT"],`	Diesel = DHT yield × DHT feed.
51–52 `"FO": M["AR"] - M["RFCC_FEED"] + M["SLURRY"],` `}`	Fuel oil = residue not sent to the RFCC + slurry.
53–54 `for p in d.products:` `M[f"MAKE_{p}"] = make[p]`	Stores each product's vector as MAKE_<product>.
55 `M["NAPH_LEFT"] = M["SR_NAPH"] - M["REF_FEED"]  # C5 >= 0`	Naphtha left after the reformer; must be ≥ 0 (constraint C5).
56 `M["AR_LEFT"] = M["AR"] - M["RFCC_FEED"]  # C7 >= 0`	Residue left after the RFCC; must be ≥ 0 (C7).
57–58 `M["RON_EXCESS"] = ((P["RON_REFORMATE"] - P["RON_SPEC_MIN"]) * M["REFORMATE"]` `+ (P["RON_FCC_GASOLINE"] - P["RON_SPEC_MIN"]) * M["FCC_GAS"])  # C9 >= 0`	Linearised gasoline RON spec (C9): Σ (RON_i − 95) × volume_i ≥ 0. Multiplying the ratio spec by pool volume keeps it linear.
59 `M["S_EXCESS"] = M["AR_S_MASS"] - P["AR_SULFUR_MAX"] * M["AR"]  # C10 <= 0`	Linearised residue sulfur spec (C10): Σ residue × (S_c − S_max) ≤ 0.
60 `M["REVENUE"] = sum(d.product.loc[p, "price_usd_bbl"] * M[f"MAKE_{p}"] for p in d.products)`	Revenue = Σ product price × product make.
61–62 `M["OPEX"] = (P["CDU_OPEX"] * M["CDU"] + P["REF_OPEX"] * M["REF_FEED"]` `+ P["RFCC_OPEX"] * M["RFCC_FEED"] + P["DHT_OPEX"] * M["DHT"])`	Variable opex = CDU, reformer, RFCC and DHT feed × their $/bbl cost.
63 `M["MARGIN"] = M["REVENUE"] - M["CRUDE_COST"] - M["OPEX"]  # $k per kbbl = $/bbl`	Gross margin = revenue − crude cost − opex. With kbbl/d × $/bbl the result is in $k/d, so this vector is the margin per barrel of each decision.
64 `return M`	Returns all vectors.
67–68 `def tank_vector(d: PlanData, M: dict, tank_id: str):` `"""(coef vector of decision-driven net inflow, fixed net inflow per day) for one tank."""`	For one tank returns (a) how decisions change its level each day and (b) the fixed daily flows (receipts or liftings).

Line · statement(s)	What it does
69 `row = d.tank.loc[tank_id]`	The tank's master data.
70 `mid = row["material_id"]`	The crude or product stored in it.
71–72 `if row["type"] == "Crude": return -np.eye(len(d.var_names))[d.crudes.index(mid)], d.R[mid].to_numpy(float)`	Crude tank: the CDU charge of that crude drains it (−1 on its x), and receipts fill it.
73 `return M[f"MAKE_{mid}"], -d.L[mid].to_numpy(float)`	Product tank: production fills it, liftings drain it.
76 `def margin_per_unit(d: PlanData) -> pd.DataFrame:`	Table of $/bbl economics for each decision, shown in the Process model block.
77 `M = coeffs(d)`	Builds the coefficient vectors.
78 `names = [f"Crude {c} ({d.crude.loc[c, 'crude_name']})" for c in d.crudes] + ["Naphtha → Reformer", "Residue → RFCC"]`	Readable decision names, e.g. "Crude AL (Arab Light)", "Naphtha → Reformer".
79–81 `return pd.DataFrame({"decision": names, "var": d.var_names, "revenue_usd_bbl": M["REVENUE"], "crude_cost_usd_bbl": M["CRUDE_COST"], "opex_usd_bbl": M["OPEX"], "margin_usd_bbl": M["MARGIN"]}).round(3)`	Returns revenue, crude cost, opex and margin per barrel. Note the text column "var"; an older app.py tried to number-format it, which caused "Unknown format code 'f' for object of type 'str'".
84–85 `def evaluate(d: PlanData, V: np.ndarray | None = None) -> dict: """Recalculate the full plan for decision matrix V (T x n). Default = decisions in the CSV."""`	Recalculates the whole plan for a decision matrix: daily plan, tank balance, constraint checks and KPIs. This is the Python version of the Excel sheets.
86 `V = d.V if V is None else np.asarray(V, float)`	Uses the decisions from the CSV unless an optimised matrix is passed in.
87 `M = coeffs(d)`	Coefficient vectors.
88 `P = d.P`	Unit parameters.
89 `q = {k: V @ vec for k, vec in M.items()}`	One matrix multiplication per quantity gives its value on every day.
91 `dp = d.decisions[["day", "date"]].copy()`	Starts the daily plan table.
92 `dp["status"] = np.where(dp["day"] < d.first_plan_day, "Actual", "Plan")`	Actual or Plan for each day.
93–94 `for i, name in enumerate(d.var_names): dp[name] = V[:, i]`	Adds the six decision columns.
95–99 `for k, col in [("CDU", "cdu_total"), ("SR_LPG", "sr_lpg"), ("SR_NAPH", "sr_naphtha"), ("SR_KERO", "sr_kero"), ("SR_GO", "sr_gasoil"), ("AR", "atm_residue"), ("REFORMATE", "reformate"), ("REF_LPG", "ref_lpg"), ("FCC_GAS", "fcc_gasoline"), ("FCC_LPG", "fcc_lpg"), ("LCO", "lco"), ("SLURRY", "slurry"), ("DHT", "dht_feed")]: dp[col] = q[k]`	Adds unit flow columns (CDU total, SR cuts, reformer, RFCC, DHT).
100 `with np.errstate(divide="ignore", invalid="ignore"):`	Suppresses divide-by-zero warnings for the ratio below.
101 `dp["residue_s_wt"] = np.where(q["AR"] > 0, q["AR_S_MASS"] / q["AR"], 0.0)`	Residue pool sulfur (wt%) = sulfur term ÷ residue volume; 0 if no residue.
102–103 `for p in d.products: dp[f"make_{p}"] = q[f"MAKE_{p}"]`	Adds one production column per product.

Line · statement(s)	What it does
`gas = q["REFORMATE"] + q["FCC_GAS"]`	Gasoline pool volume.
`with np.errstate(divide="ignore", invalid="ignore"):`	Again suppress divide warnings.
`dp["gasoline_ron"] = np.where(gas > 0, (q["REFORMATE"] * P["RON_REFORMATE"] + q["FCC_GAS"] * P["RON_FCC_GASOLINE"]) / gas, 0.0)`	Volume-weighted gasoline RON.
`dp["margin_usd_bbl"] = np.where(q["CDU"] > 0, q["MARGIN"] / q["CDU"], 0.0)`	Margin per barrel of crude processed.

`dp["revenue_kusd"] = q["REVENUE"]`
`dp["crude_cost_kusd"] = q["CRUDE_COST"]`
`dp["opex_kusd"] = q["OPEX"]`
`dp["margin_kusd"] = q["MARGIN"]`
	Daily revenue, crude cost, opex and gross margin in $k.

`tb_rows = []`	Collects one tank-balance row per tank per day.
`for tid, row in d.tank.iterrows():`	Loop over the 10 tanks.
`vec, fixed = tank_vector(d, M, tid)`	How decisions and fixed flows move this tank.
`closing = row["opening_kbbl"] + np.cumsum(V @ vec + fixed)`	Closing stock = opening + running total of (inflow − outflow).
`opening = np.r_[row["opening_kbbl"], closing[:-1]]`	Each day's opening = previous day's closing (day 1 uses the master opening).
`inflow, outflow = (fixed, -(V @ vec)) if row["type"] == "Crude" else (V @ vec, -fixed)`	Crude tank: in = receipts, out = CDU charge. Product tank: in = production, out = liftings.
`for t in range(d.T):`	Loop over days.
`c = closing[t]`	Closing stock on day t.
`status = "OK" if row["min_heel_kbbl"] - TOL <= c <= row["capacity_kbbl"] + TOL else ("LOW" if c < row["min_heel_kbbl"] else "HIGH")`	OK if within [min heel, capacity], LOW below heel, HIGH above capacity.
`tb_rows.append({"tank_id": tid, "service": row["service"], "type": row["type"], "day": t + 1, "date": d.decisions["date"].iat[t], "opening": opening[t], "inflow": inflow[t], "outflow": outflow[t], "closing": c, "min_heel": row["min_heel_kbbl"], "capacity": row["capacity_kbbl"], "fill_pct": c / row["capacity_kbbl"], "status": status ↪ })`	Adds the row: opening, inflow, outflow, closing, limits, fill % and status.
`tb = pd.DataFrame(tb_rows)`	Tank balance table (long format, like the Tank_Balance sheet).
`xmax, xmin = d.crude["x_max_kbd"].to_numpy(float), d.crude["x_min_kbd"].to_numpy(float)`	Crude rate limits.
`X = V[:, :len(d.crudes)]`	The crude-rate part of the decisions.
`cdu = q["CDU"]`	Daily CDU throughput.
`ramp = np.r_[0.0, np.diff(cdu)]`	Day-to-day CDU change (0 on day 1).
`tank_breach = tb.groupby("day")["status"].apply(lambda s: (s != "OK").sum()).to_numpy()`	Number of tanks out of range on each day.
`ck = dp[["day", "date", "status"]].copy()`	Starts the constraint-check table (True = OK).
`ck["C1"] = (cdu >= P["CDU_MIN"] - TOL) & (cdu <= P["CDU_MAX"] + TOL)`	C1 CDU within min/max.
`ck["C2"] = ((X >= xmin - TOL) & (X <= xmax + TOL)).all(axis=1)`	C2 every crude within its rate limits.

Line · statement(s)	What it does
138 `ck["C3"] = np.abs(ramp) <= P["CDU_RAMP"] + TOL`	C3 CDU ramp within limit.
139 `ck["C4"] = (q["REF_FEED"] >= P["REF_MIN"] - TOL) & (q["REF_FEED"] <= P["REF_MAX"] + TOL)`	C4 reformer within min/max.
140 `ck["C5"] = q["NAPH_LEFT"] >= -TOL`	C5 enough naphtha for the reformer.
141 `ck["C6"] = (q["RFCC_FEED"] >= P["RFCC_MIN"] - TOL) & (q["RFCC_FEED"] <= P["RFCC_MAX"] + TOL)`	C6 RFCC within min/max.
142 `ck["C7"] = q["AR_LEFT"] >= -TOL`	C7 enough residue for the RFCC.
143 `ck["C8"] = (q["DHT"] >= P["DHT_MIN"] - TOL) & (q["DHT"] <= P["DHT_MAX"] + TOL)`	C8 DHT within min/max.
144 `ck["C9"] = q["RON_EXCESS"] >= -TOL`	C9 gasoline RON ≥ spec.
145 `ck["C10"] = q["S_EXCESS"] <= TOL`	C10 residue sulfur ≤ spec.
146 `ck["C13"] = tank_breach == 0`	C13 all tanks within window.
147 `flags = ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8", "C9", "C10", "C13"]`	List of the check columns.
148 `ck["violations"] = (~ck[flags]).sum(axis=1)`	Counts failed checks per day.
149 `ck["day_result"] = np.where(ck["violations"] == 0, "FEASIBLE", "VIOLATION")`	FEASIBLE or VIOLATION per day.
151 `last = tb[tb["day"] == d.T].set_index("tank_id")`	Tank rows for the last day.

152–154
`c14 = pd.DataFrame({"tank_id": d.tank.index, "service": d.tank["service"].values,`
`"closing_last_day": last.loc[d.tank.index, "closing"].values,`
`"end_target": d.tank["end_target_kbbl"].values})`
	End-of-horizon table: closing stock vs target per tank.

155 `c14["C14_ok"] = c14["closing_last_day"] >= c14["end_target"] - TOL`	C14 closing ≥ target.
157 `plan = dp["status"] == "Plan"`	Marks plan days.

158–162
`kpi = {"month_margin_kusd": float(dp["margin_kusd"].sum()),`
`"actual_margin_kusd": float(dp.loc[~plan, "margin_kusd"].sum()),`
`"plan_margin_kusd": float(dp.loc[plan, "margin_kusd"].sum()),`
`"crude_run_kbbl": float(cdu.sum()),`
`"violations": int(ck["violations"].sum() + (~c14["C14_ok"]).sum())}`
	KPIs: month margin, locked actual-days margin, plan-days margin (the objective), total crude run, total violations.

163 `return {"daily_plan": dp, "tank_balance": tb, "checks": ck, "c14": c14, "kpi": kpi}`	Returns all result tables.

4. optimizer.py – LP optimizer

Builds and solves the linear program (SciPy HiGHS) and produces binding constraints, shadow prices and plain-language reasons. Unchanged in rev. 2.

Line · statement(s)	What it does

1–6
`"""LP optimizer: maximise plan-days gross margin subject to C1-C16 (SciPy HiGHS, no licence needed).`
 	
`Decision vector per plan day t: [x_c1..x_cn, r, f] (kbbl/d).`
`Soft mode adds penalised slack on tank min-heel / end-of-month targets so infeasible`
`scenarios still return a plan plus the size of each shortfall.`
`"""`
	Module description: builds and solves the linear program that maximises plan-days gross margin, then explains the result.

7 `from __future__ import annotations`	Modern type hints.
9 `from dataclasses import dataclass, field`	dataclass/field for the result container.

11–12
`import numpy as np`
`import pandas as pd`
	numpy and pandas.

13 `from scipy.optimize import linprog`	SciPy's linear-programming solver. method="highs" uses the free HiGHS engine; no licence needed.
15 `from data_loader import PlanData`	Input container…
16 `from model import coeffs, tank_vector`	…and the same coefficient vectors used for plan evaluation, so the optimizer and the checks always agree.

18–23
`CONSTRAINT_LABEL = {`
`"C1": "CDU capacity", "C2": "Crude rate limit", "C3": "CDU ramp rate", "C4": "Reformer capacity",`
`"C5": "Naphtha availability", "C6": "RFCC capacity", "C7": "Residue availability", "C8": "DHT capacity",`
`"C9": "Gasoline RON spec", "C10": "Residue sulfur spec", "C13": "Tank operating window",`
`"C14": "End-of-month inventory",`
`}`
	Readable names for constraint IDs, used in the explanations.

26–27
`@dataclass`
`class OptResult:`
	OptResult: everything the solver returns.

28 `success: bool`	True if an optimal plan was found.
29 `message: str`	Solver message ("Optimal", "infeasible", …).
30 `V: np.ndarray`	Full T × 6 decision matrix (actual days unchanged, plan days optimised).

31–32
`plan_margin_kusd: float = float("nan")`
`penalty_kusd: float = 0.0`
	Plan-days margin and, in soft mode, the penalty paid for tank shortfalls.

33–34
`binding: pd.DataFrame = field(default_factory=pd.DataFrame)`
`shortfalls: pd.DataFrame = field(default_factory=pd.DataFrame)`
	Constraints at their limit with shadow prices, and tank shortfalls (soft mode).

35–36
`n_rows: int = 0`
`n_vars: int = 0`
	Problem size: inequality rows and variables.

39–40
`class _LP:`
`"""Collects A_ub x <= b_ub rows, each with a readable label."""`
	Small helper that collects the constraint rows "A·x ≤ b", each with a readable label.

42 `def __init__(self, n: int, days: list[int]):`	Set-up:

Line · statement(s)	What it does
43 `self.n, self.days = n, days`	n = variables per day (6); days = plan days.
44 `self.n_dec = n * len(days)`	Total decision variables (6 × 20 = 120 in the base case).
45 `self.n_slack = 0`	Number of slack (shortfall) variables added in soft mode.
46 `self.rows, self.rhs, self.meta = [], [], []`	Row terms, right-hand sides, and labels (constraint, day, item, side).
48 `def col(self, day: int) -> int:`	Column where a given day's 6 variables start.
49 `return (day - self.days[0]) * self.n`	e.g. first plan day → column 0, next day → column 6.
51–53 `def new_slack(self) -> int:` … `return self.n_dec + self.n_slack - 1`	Adds one slack variable and returns its column (after all decision columns).
55–58 `def le(self, terms, rhs, cid, day, item, side, slack=None):` … `self.meta.append({"constraint": cid, "day": day, "item": item, "side": side})`	Adds one constraint "Σ (vector · day's variables) − slack ≤ rhs" with its label.
60 `def matrix(self):`	Builds the dense matrix A and vector b for the solver.
61 `A = np.zeros((len(self.rows), self.n_dec + self.n_slack))`	Empty matrix: rows × (decisions + slacks).
62 `for i, (terms, slack) in enumerate(self.rows):`	For each stored row…
63–65 `for day, vec in terms:` … `A[i, c:c + self.n] += vec`	…writes each day's coefficient vector into that day's columns…
66–67 `if slack is not None:` … `A[i, slack] -= 1.0`	…and −1 in its slack column (slack relaxes the limit).
68 `return A, np.array(self.rhs, float)`	Returns A and b.
71 `def optimize(d: PlanData, soft_tanks: bool = False, penalty_usd_bbl: float = 1000.0) -> OptResult:`	Builds and solves the LP. soft_tanks lets tank min-heel/end targets be missed at a penalty ($/bbl).
72 `M, P = coeffs(d), d.P`	Coefficient vectors and parameters.
73 `V_hist = d.V.copy()`	Current decisions; actual days stay fixed (C15).
74 `fp, T = d.first_plan_day, d.T`	First plan day and horizon length.
75 `days = list(range(fp, T + 1))`	Days the optimizer may change.
76–77 `if not days:` … `return OptResult(False, "No Plan days left in the horizon - nothing to optimise.", V_hist)`	No plan days → nothing to optimise.
78 `n = len(d.var_names)`	Variables per day.
79 `lp = _LP(n, days)`	Empty LP.
81 `for t in days:`	Daily unit constraints:
82 `lp.le([(t, M["CDU"])], P["CDU_MAX"], "C1", t, "CDU", "max")`	C1 CDU ≤ max…

Line · statement(s)	What it does
83 `lp.le([(t, -M["CDU"])], -P["CDU_MIN"], "C1", t, "CDU", "min")`	…and CDU ≥ min (written as −CDU ≤ −min).
84 `lp.le([(t, -M["NAPH_LEFT"])], 0.0, "C5", t, "SR naphtha", "max")`	C5 reformer feed ≤ available naphtha.
85 `lp.le([(t, -M["AR_LEFT"])], 0.0, "C7", t, "Atm residue", "max")`	C7 RFCC feed ≤ available residue.
86 `lp.le([(t, M["DHT"])], P["DHT_MAX"], "C8", t, "DHT", "max")`	C8 DHT ≤ max…
87 `lp.le([(t, -M["DHT"])], -P["DHT_MIN"], "C8", t, "DHT", "min")`	…and ≥ min.
88 `lp.le([(t, -M["RON_EXCESS"])], 0.0, "C9", t, "Gasoline 95", "min")`	C9 gasoline RON ≥ spec.
89 `lp.le([(t, M["S_EXCESS"])], 0.0, "C10", t, "Residue pool", "max")`	C10 residue sulfur ≤ spec.
90 `if t == fp and fp > 1:  # C3 linked to last actual day`	C3 ramp on the first plan day is measured against the last actual day…
91 `prev = V_hist[fp - 2] @ M["CDU"]`	…whose CDU rate is a fixed number…
92–93 `lp.le([(t, M["CDU"])], P["CDU_RAMP"] + prev, "C3", t, "CDU", "up")` `lp.le([(t, -M["CDU"])], P["CDU_RAMP"] - prev, "C3", t, "CDU", "down")`	…so CDU_t ≤ prev + ramp and CDU_t ≥ prev − ramp.

94–96 `elif t > fp:`
`lp.le([(t, M["CDU"]), (t - 1, -M["CDU"])], P["CDU_RAMP"], "C3", t, "CDU", "up")`
`lp.le([(t, -M["CDU"]), (t - 1, M["CDU"])], P["CDU_RAMP"], "C3", t, "CDU", "down")`
	Later days: |CDU_t − CDU_(t−1)| ≤ ramp, written as two rows linking two days.

98 `# C11/C12 balances substituted into C13/C14:  I[k,t] = base[t] + sum_{s=fp..t} vec . x_s`	Tank constraints. The balance I_t = opening + flows is substituted directly, so no inventory variables are needed.
99 `for tid, row in d.tank.iterrows():`	For each tank:
100 `vec, fixed = tank_vector(d, M, tid)`	Its decision vector and fixed daily flows.
101 `hist = np.r_[V_hist[:fp - 1] @ vec, np.zeros(T - fp + 1)]`	Net flow caused by the locked actual days (zero for plan days).
102 `base = row["opening_kbbl"] + np.cumsum(fixed + hist)`	Stock each day if all plan-day decisions were zero: opening + receipts/liftings + actual-day effect.
103 `for t in days:`	For each plan day:
104 `lp.le([(s, vec) for s in range(fp, t + 1)], row["capacity_kbbl"] - base[t - 1], "C13", t, tid, "max")`	C13 base + decision effect up to day t ≤ capacity.
105–106 `lp.le([(s, -vec) for s in range(fp, t + 1)], base[t - 1] - row["min_heel_kbbl"], "C13", t, tid, "min",` `slack=lp.new_slack() if soft_tanks else None)`	C13 stock ≥ min heel (with a slack in soft mode).
107–108 `lp.le([(s, -vec) for s in days], base[T - 1] - row["end_target_kbbl"], "C14", T, tid, "min",` `slack=lp.new_slack() if soft_tanks else None)`	C14 last-day stock ≥ end target (with a slack in soft mode).
110–111 `day_bounds = ([(d.crude.loc[c, "x_min_kbd"], d.crude.loc[c, "x_max_kbd"]) for c in d.crudes]` `+ [(P["REF_MIN"], P["REF_MAX"]), (P["RFCC_MIN"], P["RFCC_MAX"])])`	Simple bounds per day: C2 crude min/max, C4 reformer min/max, C6 RFCC min/max.
112 `bounds = day_bounds * len(days) + [(0, None)] * lp.n_slack`	Repeated for every plan day; slacks are ≥ 0 (C16 non-negativity).
113 `c = np.r_[np.tile(-M["MARGIN"], len(days)), np.full(lp.n_slack, penalty_usd_bbl)]`	Objective. linprog minimises, so margin is negated: minimise −margin + penalty × shortfall.
114 `A, b = lp.matrix()`	Builds the constraint matrix.

Line · statement(s)	What it does
115 `res = linprog(c, A_ub=A, b_ub=b, bounds=bounds, method="highs")`	Solves the LP with HiGHS.
116 `if res.status != 0:` 117 `hint = "" if soft_tanks else " Turn on 'soft tank limits' to see which tank target cannot be met."` 118 `return OptResult(False, f"{res.message}{hint}", V_hist, n_rows=A.shape[0], n_vars=A.shape[1])`	Not optimal (usually infeasible) → return the reason and a hint to use soft mode.
120 `x = res.x`	Solution vector.
121 `V = V_hist.copy()`	Starts from current decisions…
122 `V[fp - 1:] = x[:lp.n_dec].reshape(len(days), n)`	…and replaces the plan days with the optimised values.
123 `plan_margin = float((V[fp - 1:] @ M["MARGIN"]).sum())`	Plan-days margin of the optimised plan.
124 `penalty = float(penalty_usd_bbl * x[lp.n_dec:].sum()) if lp.n_slack else 0.0`	Total shortfall penalty (soft mode only).
126 `meta = pd.DataFrame(lp.meta)`	Explainability: one row per constraint with its label…
127 `meta["slack"] = res.ineqlin.residual`	…remaining room to the limit…
128 `meta["shadow_price"] = -res.ineqlin.marginals`	…and shadow price: extra margin ($k) if the limit were relaxed by one unit.
129 `binding = meta[meta["slack"] < 1e-6].copy()`	Binding = constraints with no room left.
130 `names = [f"Crude {c}" for c in d.crudes] + ["Reformer", "RFCC"]`	Labels for variable bounds…
131 `cids = ["C2"] * len(d.crudes) + ["C4", "C6"]`	…and their constraint IDs.
132 `extra = []`	Bounds hit by the solution are added to the binding list:
133 `for k, t in enumerate(days):` 134 `for j in range(n):`	For every plan day and variable…
135 `i = k * n + j`	…position in the solution vector…
136 `lo, hi = day_bounds[j]`	…its bounds…
137 `for side, hit, mu in [("max", x[i] >= hi - 1e-6, -res.upper.marginals[i]),` 138 `("min", x[i] <= lo + 1e-6, res.lower.marginals[i])]:`	…check whether it sits at max or min, with the matching shadow price…
139 `if hit and not (side == "min" and lo == 0 and j < len(d.crudes)):`	…ignore crudes at zero (that just means "not used")…
140 `extra.append({"constraint": cids[j], "day": t, "item": names[j], "side": side,` 141 `"slack": 0.0, "shadow_price": mu})`	…record the bound as binding.
142 `binding = pd.concat([binding, pd.DataFrame(extra)], ignore_index=True)`	Combines row and bound constraints.
143 `binding["label"] = binding["constraint"].map(CONSTRAINT_LABEL)`	Adds readable names.
145 `sf = []`	Soft mode: list each tank shortfall:
146 `if lp.n_slack:`	Only when slack variables exist.
147 `for i, (_, slack) in enumerate(lp.rows):`	Rows that have a slack…
148 `if slack is not None and x[slack] > 1e-6:`	…with a positive value are shortfalls…

Line · statement(s)	What it does
149 `m = lp.meta[i]`	…taken with their label…
150–151 `sf.append({"constraint": m["constraint"], "day": m["day"], "tank_id": m["item"], "shortfall_kbbl": float(x[slack])})`	…and stored (constraint, day, tank, kbbl short).
152–153 `return OptResult(True, "Optimal", V, plan_margin, penalty, binding, pd.DataFrame(sf), n_rows=A.shape[0], n_vars=A.shape[1])`	Returns the full result.
156 `def binding_summary(d: PlanData, res: OptResult) -> pd.DataFrame:`	Groups binding rows by constraint and item across days.
157–158 `if res.binding.empty: return pd.DataFrame(columns=["constraint", "label", "item", "side", "days", "value"])`	Nothing binding → empty table.
159–161 `return (res.binding.groupby(["constraint", "label", "item", "side"]) .agg(days=("day", "nunique"), value=("shadow_price", "sum")).reset_index() .sort_values("value", key=np.abs, ascending=False))`	Counts days and sums shadow prices, sorted by size of the shadow price.
164–165 `def explain(d: PlanData, base_kpi: dict, opt_kpi: dict, res: OptResult) -> list[str]: """Plain-language reasons built from binding constraints and shadow prices."""`	Turns solver results into plain sentences.
166–168 `out = [f"Plan-days margin moves from {base_kpi['plan_margin_kusd']:,.0f} $k (current plan) to " f"{opt_kpi['plan_margin_kusd']:,.0f} $k (optimised): " f"{opt_kpi['plan_margin_kusd'] - base_kpi['plan_margin_kusd']:+,.0f} $k."]`	First line: current vs optimised margin and the uplift.
169 `n_days = d.T - d.first_plan_day + 1`	Number of plan days.
170 `for _, r in binding_summary(d, res).head(8).iterrows():`	Top 8 limiting constraints:
171–172 `txt = (f"**{r['label']} ({r['constraint']}) — {r['item']} {r['side']}** is limiting on " f"{r['days']} of {n_days} plan days")`	"<constraint> is limiting on N of M plan days"…
173–174 `if abs(r["value"]) >= 0.05: txt += f"; relaxing it by one unit on those days would add about {abs(r['value']):,.1f} $k."`	…plus its value if relaxed…
175–176 `else: txt += " (at the limit, but relaxing it adds no margin)."`	…or a note that relaxing it adds nothing.
177 `out.append(txt)`	Adds the sentence.
178–180 `for _, r in res.shortfalls.iterrows(): out.append(f"Tank **{r['tank_id']}** misses its {CONSTRAINT_LABEL[r['constraint']].lower()} on day " f"{r['day']} by {r['shortfall_kbbl']:,.1f} kbbl — this scenario is not fully feasible.")`	One sentence per tank shortfall (soft mode).
181 `return out`	Returns the sentences.

5. app.py – Web host

Streamlit page: loads data, validates, decides whether to solve, sends results to the canvas and processes user actions. REVISED in rev. 2 – changed statements are tagged [NEW] / [REVISED] and shaded.

Line · statement(s)	What it does
1 `"""Refinery Production Planning Optimizer — interactive workflow canvas (Streamlit host).` 2 3 `    streamlit run app.py` 4 5 `The canvas (flow_component/index.html) is a two-way Streamlit component:` 6 `  Python -> canvas : tables, validation, model coefficients, optimizer results (JSON "payload")` 7 `  canvas -> Python : user actions {nonce, action, ...}` 8 9 `Run behaviour:` 10 `  * "Apply" on an input block only saves the edited table and re-validates it.` 11 `    Outputs keep showing the LAST optimizer run and are flagged "re-run needed".` 12 `  * The LP is solved only on: first load, "Run optimizer", or applying run settings.` 13 `  * Solved results are cached per input fingerprint, so undoing an edit (or Reset)` 14 `    brings back the matching results instantly.` 15 `"""`	Module description. **[REVISED]** Now also documents the run behaviour: Apply only saves and validates an input table; the LP is solved only on first load, ▶ Run optimizer, or Apply & solve in Run settings; results are cached per input fingerprint.
16 `from __future__ import annotations`	Modern type hints.
18 `import io` 19 `import math` 20 `import zipfile` 21 `from pathlib import Path`	io (in-memory zip files), math (NaN checks), zipfile (download bundles), Path (component folder).
23 `import numpy as np` 24 `import pandas as pd` 25 `import streamlit as st` 26 `import streamlit.components.v1 as components`	numpy, pandas, Streamlit, and Streamlit's custom-component API.
28 `import data_loader` 29 `from model import coeffs, evaluate, margin_per_unit` 30 `from optimizer import binding_summary, explain, optimize`	Engine modules: loader, model functions and optimizer functions.
32 `DEFAULT_SOURCE = "https://github.com/iqbalshafiq96/Self-Work/tree/main/Refinery_Optimizer"`	Default GitHub folder to load the CSVs from.
33 `INPUT_TABLES = ["crude_data", "unit_data", "product_data", "tank_master",` 34 `  "crude_receipts", "product_liftings", "daily_decisions"]`	The seven editable input tables (config is not editable on the canvas).

Line · statement(s)	What it does
35 TABLE_META = { 36 "crude_data": dict(readonly=["crude_id", "tank_id"], 37 desc="Delivered price, CDU cut yields (must sum to 1.000), residue sulfur and charge limits p ↪ er crude."), 38 "unit_data": dict(readonly=["param_key", "symbol", "unit", "parameter", "uom", "used_in", "comment"], 39 desc="Unit capacities, ramp limit, variable opex, conversion-unit yields and blend specs. Edit ↪ the 'value' column."), 40 "product_data": dict(readonly=["product_id", "tank_id", "make_formula", "produced_from"], 41 desc="Product netback prices. 'make_formula' shows how each product is made from x, r, f.") ↪ , 42 "tank_master": dict(readonly=["tank_id", "service", "material_id", "type"], 43 desc="Tank working capacity, minimum heel, opening inventory (day 1) and end-of-horizon targ ↪ et."), 44 "crude_receipts": dict(readonly=["receipt_id", "date", "tank_id"], dynamic=True, 45 desc="Crude cargo arrivals. Change 'day' to simulate a vessel delay; add or delete cargoe ↪ s."), 46 "product_liftings": dict(readonly=["lifting_id", "date", "tank_id"], dynamic=True, 47 desc="Product liftings / cargoes. Change 'day' or 'qty_kbbl' for demand scenarios."), 48 "daily_decisions": dict(readonly=["day", "date", "status"], 49 desc="Recorded rates. Rows before the first plan day are locked actuals; Plan rows are t ↪ he current (manual) plan used as the comparison baseline."), 50 }	Per table: which columns are read-only, whether rows can be added/deleted ("dynamic"), and the help text shown in the editor.
51 OUTPUT_SUMMARY_KEYS = ["model", "optimizer", "decisions", "economics", "tank_balance", "tank_curves", "why"]	**[NEW]** Blocks whose one-line summary comes from the last optimizer run (model, optimizer and the five outputs). Input-block summaries always come from the current inputs.
53 st.set_page_config(page_title="Refinery Planning Flow", page_icon="[oil-drum emoji]", layout="wide", 54 initial_sidebar_state="collapsed")	Browser tab title and icon, wide layout, sidebar collapsed.
55 st.markdown("<style>.block-container{padding-top:2.6rem;padding-bottom:1rem;max-width:100%}</style>", 56 unsafe_allow_html=True)	Small CSS tweak: page padding and full width.
57 flow_canvas = components.declare_component("flow_canvas", path=str(Path(__file__).parent / "flow_component"))	Registers flow_component/index.html as a two-way component called flow_canvas.
61 def clean(o): 62 """Make any nested structure JSON-safe (numpy/pandas -> python, NaN/inf -> None)"""	Makes any data safe to send as JSON to the browser:
63 if isinstance(o, dict): 64 return {str(k): clean(v) for k, v in o.items()}	dictionaries → cleaned recursively, keys as text;
65 if isinstance(o, (list, tuple)): 66 return [clean(v) for v in o]	lists → cleaned item by item;
67 if isinstance(o, (np.ndarray, pd.Series, pd.Index)): 68 return clean(list(o.tolist()))	numpy arrays and pandas series → plain lists;
69 if isinstance(o, pd.DataFrame): 70 return clean(o.to_dict("records"))	tables → list of row dictionaries;
71 if isinstance(o, pd.Timestamp): 72 return o.isoformat()	dates → ISO text;
73 if isinstance(o, (np.bool_, bool)): 74 return bool(o)	booleans → True/False;
75 if isinstance(o, (np.integer,)): 76 return int(o)	numpy integers → int;

Line · statement(s)	What it does
77 `if isinstance(o, (float, np.floating)):` 78 `f = float(o)` 79 `return None if math.isnan(f) or math.isinf(f) else round(f, 6)`	floats → rounded to 6 decimals; NaN or infinity → None (JSON has no NaN).
80 `return o`	Anything else (text) unchanged.
83 `def table_payload(name, df, crudes, products):`	Packs one input table for the editor:
84 `meta = TABLE_META[name]`	its settings;
85 `selects = {}` 86 `if name == "crude_receipts":` 87 `selects = {"crude_id": crudes}` 88 `elif name == "product_liftings":` 89 `selects = {"product_id": products}`	drop-down lists: crude_id in receipts, product_id in liftings;
90 `numeric = [c for c in df.columns if pd.api.types.is_numeric_dtype(df[c])]` 91 `if name == "unit_data":` 92 `numeric = ["value"]`	which columns are numeric (only "value" in unit_data);
93 `return {"columns": list(df.columns), "rows": clean(df.astype(object).values.tolist()),` 94 `"readonly": meta["readonly"], "dynamic": meta.get("dynamic", False), "selects": selects,` 95 `"numeric": numeric, "desc": meta["desc"], "file": data_loader.FILES[name]}`	columns, rows, read-only list, dynamic flag, drop-downs, numeric list, description and file name.
98 `def frames_equal(a: pd.DataFrame, b: pd.DataFrame) -> bool:` 99 `try:` 100 `return a.shape == b.shape and a.astype(str).reset_index(drop=True).equals(b.astype(str).reset_index(drop=Tru e))` 101 `except Exception:` 102 `return False`	True if two tables are identical (compared as text). Used to mark blocks "EDITED".
105 `def rows_to_frame(name, columns, rows, frames):` 106 `"""Convert edited rows from the canvas back into a typed DataFrame."""`	Converts rows sent by Apply back into a typed DataFrame.
107 `df = pd.DataFrame(rows, columns=columns)`	Builds the table.
108 `orig = st.session_state.base_frames[name]`	Original table, used to know which columns are numeric.
109 `for c in df.columns:` 110 `if c in orig.columns and pd.api.types.is_numeric_dtype(orig[c]) or (name == "unit_data" and c == "value"):` 111 `df[c] = pd.to_numeric(df[c].replace("", np.nan), errors="coerce")`	Numeric columns: blanks → NaN, text → NaN (validation will flag them).
112 `if name in ("crude_receipts", "product_liftings"):`	For cargo tables:
113 `idcol, prefix = ("crude_id", "R") if name == "crude_receipts" else ("product_id", "L")`	which ID column and ID prefix (R/L);
114 `key = "receipt_id" if name == "crude_receipts" else "lifting_id"`	ID column name;
115 `tank_of = dict(zip(frames["tank_master"]["material_id"], frames["tank_master"]["tank_id"]))`	material → tank lookup;
116 `dates = dict(zip(pd.to_numeric(frames["daily_decisions"]["day"]), frames["daily_decisions"]["date"]))`	day → date lookup;
117 `df = df.sort_values("day", kind="stable").reset_index(drop=True)`	sort by day;
118 `df[key] = [f"{prefix}{i + 1:03d}" for i in range(len(df))]`	renumber IDs R001, R002…;
119 `if "tank_id" in df.columns:` 120 `df["tank_id"] = df[idcol].map(tank_of)`	refill tank_id from the material…

Line · statement(s)	What it does

121
122
`if "date" in df.columns:`
`df["date"] = df["day"].map(dates)`
	…and date from the day, so read-only columns stay consistent.

123 `return df`	Returns the typed table.

126
127
128
129
`def default_first_plan_day(frames):`
`dec = frames["daily_decisions"]`
`plan = dec.loc[dec["status"] == "Plan", "day"]`
`return int(plan.min()) if len(plan) else len(dec)`
	First day whose status is "Plan" in the original data.

132 `def load_source(url):`	Loads all CSVs from a URL and resets the session:
133 `frames, sources = data_loader.read_csvs_from_url(url)`	download (with local fallback);

134
135
`st.session_state.base_frames = {k: v.copy() for k, v in frames.items()}`
`st.session_state.frames = {k: v.copy() for k, v in frames.items()}`
	base_frames = untouched copy (for Reset); frames = working copy that Apply changes;

136 `st.session_state.sources = sources`	where each file came from;
137 `st.session_state.settings = {"first_plan_day": default_first_plan_day(frames), "soft": False, "penalty": 1000.0}`	default settings: first plan day, soft mode off, penalty 1000 $/bbl;
138 `st.session_state.solve_cache = {}  # fingerprint -> results bundle`	empty results cache (input fingerprint → results bundle);
139 `st.session_state.last_bundle = None  # results of the most recent optimizer run`	**[NEW]** results of the most recent optimizer run – shown while inputs are changed but not yet re-run;
140 `st.session_state.run_requested = True  # solve once on first load`	**[NEW]** flag that tells the next script run to solve; True so the app solves once on first load.

143
144
`def make_bundle(data, base, res):`
`"""Everything the output blocks need from ONE optimizer run (kept until the next run)."""`
	[NEW] Packs everything the output blocks need from ONE optimizer run (model data, results, block summaries, download zip). The bundle is kept until the next run, so outputs can still be shown after inputs change.

145 `opt = evaluate(data, res.V) if res.success else None`	Evaluates the optimised plan if the solve succeeded…
146 `view = opt or base`	…and shows it, or the current plan if no solution.

147
148
149
150
`M = coeffs(data)`
`mpu = margin_per_unit(data)`
`P = data.P`
`crudes, products = data.crudes, data.products`
	Coefficient vectors, margin-per-barrel table, parameters, crude/product lists.

152
153
154
155
156
157
158
159
160
161
162
163
164
`model = {`
`"var_names": data.var_names, "crudes": crudes,`
`"crude_names": data.crude["crude_name"].tolist(),`
`"products": products, "product_names": data.product["product_name"].tolist(),`
`"P": {k: float(v) for k, v in P.items()},`
`"crude": data.crude.reset_index().to_dict("records"),`
`"margin_per_unit": mpu.to_dict("records"),`
`"make": {p: M[f"MAKE_{p}"] for p in products},`
`"dht": M["DHT"], "sulfur_row": M["S_EXCESS"], "ron_row": M["RON_EXCESS"],`
`"n_rows": res.n_rows, "n_vars": res.n_vars,`
`"tanks": data.tank.reset_index()[["tank_id", "service", "type", "capacity_kbbl", "min_heel_kbbl",`
`"opening_kbbl", "end_target_kbbl"]].to_dict("records"),`
`}`
	Model section for the calculation blocks: variable names, crudes, products, parameters, margin per barrel, make vectors, spec rows, LP size, tank master data.

Line · statement(s)	What it does
166 `def tank_series(ev):` 167 `    return {tid: g.sort_values("day") for tid, g in ev["tank_balance"].groupby("tank_id")}`	Splits a tank balance into one sorted series per tank.
169 `tb_base, tb_view = tank_series(base), tank_series(view)`	Tank series for the current and the displayed plan.
170 `tanks = []` 171 `for tid, row in data.tank.iterrows():` 172 `    g, gb = tb_view[tid], tb_base[tid]` 173 `    tanks.append({"id": tid, "service": row["service"], "type": row["type"],` 174 `                  "cap": row["capacity_kbbl"], "heel": row["min_heel_kbbl"], "target": row["end_target_kbbl"],` 175 `                  "opening": g["opening"].tolist(), "inflow": g["inflow"].tolist(), "outflow": g["outflow"].toli st(),` 176 `                  "closing": g["closing"].tolist(), "closing_base": gb["closing"].tolist(),` 177 `                  "status": g["status"].tolist()})`	One entry per tank for the tank blocks: limits, target, opening/inflow/outflow/closing, current-plan closing and status per day.
178 `dpv, dpb = view["daily_plan"], base["daily_plan"]`	Displayed and current daily plans.
179 `plan_mask = dpb["status"] == "Plan"`	Plan-day rows.
180 `econ_cols = ["revenue_kusd", "crude_cost_kusd", "opex_kusd", "margin_kusd"]`	Economics columns to total.
181 `bsum = binding_summary(data, res) if res.success else pd.DataFrame()`	Grouped binding constraints.
182 `check_cols = ["C1", "C2", "C3", "C4", "C5", "C6", "C7", "C8", "C9", "C10", "C13"]`	Constraint-check columns.
183 `result = {` 184 `    "success": res.success, "message": res.message,` 185 `    "days": data.decisions["day"].tolist(), "dates": data.decisions["date"].astype(str).tolist(),` 186 `    "status": dpb["status"].tolist(), "first_plan_day": data.first_plan_day,` 187 `    "V_base": np.round(dpb[data.var_names].to_numpy(float), 3),` 188 `    "V_opt": np.round(dpv[data.var_names].to_numpy(float), 3),` 189 `    "kpi_base": base["kpi"], "kpi_opt": opt["kpi"] if opt else None,` 190 `    "benchmark": float(data.cfg["benchmark_lp_optimum_kusd"]) if "benchmark_lp_optimum_kusd" in data.cfg else No ne,` 191 `    "penalty_kusd": res.penalty_kusd,` 192 `    "margin_base": dpb["margin_kusd"].round(2), "margin_opt": dpv["margin_kusd"].round(2),` 193 `    "cdu_opt": dpv["cdu_total"].round(2),` 194 `    "ron_opt": dpv["gasoline_ron"].round(2), "sulfur_opt": dpv["residue_s_wt"].round(3),` 195 `    "econ": {"labels": ["Product revenue", "Crude cost", "Opex", "Gross margin"],` 196 `             "base": dpb.loc[plan_mask, econ_cols].sum().tolist(),` 197 `             "opt": dpv.loc[plan_mask, econ_cols].sum().tolist()},` 198 `    "make": [{"id": p, "name": data.product.loc[p, "product_name"],` 199 `              "price": data.product.loc[p, "price_usd_bbl"],` 200 `              "base": dpb.loc[plan_mask, f"make_{p}"].sum(), "opt": dpv.loc[plan_mask, f"make_{p}"].sum()}` 201 `             for p in products],` 202 `    "tanks": tanks,` 203 `    "c14": view["c14"].to_dict("records"),` 204 ` "checks": {c: int((~view["checks"][c]).sum()) for c in check_cols}	{` 205 ` "C14": int((~view["c14"]["C14_ok"]).sum())},` 206 ` "binding": bsum.to_dict("records"),` 207 ` "binding_count": res.binding.groupby("constraint").size().to_dict() if res.success and not res.binding.empty else {},` 208 ` "explain": explain(data, base["kpi"], opt["kpi"], res) if opt else [res.message],` 209 ` "shortfalls": res.shortfalls.to_dict("records"),` 210 `}`	Results section: status, days/dates, decisions (current and optimised), KPIs, benchmark, penalty, daily margin, CDU, RON, sulfur, economics totals, product make, tanks, C14, breach counts, binding list, explanation text and shortfalls.
212 `top = mpu.sort_values("margin_usd_bbl", ascending=False).iloc[0]`	Highest-margin decision (for the Process model summary).
213 `tk = data.tank`	Tank table for the tank-balance summary.

Line · statement(s)	What it does
214 summary = { 215 "model": f"Best: {top['var']} {top['margin_usd_bbl']:.2f} $/bbl", 216 "optimizer": f"Z = {opt['kpi']['plan_margin_kusd']:,.0f} $k" if opt else "No feasible plan", 217 "decisions": f"{(data.T - data.first_plan_day + 1) * len(data.var_names)} variables solved" if opt else "Sho ↪ wing current plan", 218 "economics": (f"{opt['kpi']['plan_margin_kusd'] - base['kpi']['plan_margin_kusd']:+,.0f} $k vs current plan" 219 if opt else f"{base['kpi']['plan_margin_kusd']:,.0f} $k current"), 220 "tank_balance": f"{int((~view['checks']['C13']).sum())} breach day(s) · C14 {int(view['c14']['C14_ok'].sum() ↪)}/{len(tk)} OK", 221 "tank_curves": "Closing inventory vs limits", 222 "why": (bsum.iloc[0]["label"] + " is the top constraint") if len(bsum) else res.message[:60], 223 }	One-line text for the model, optimizer and output blocks (e.g. "Z = 18,320 $k").
225 downloads = None 226 if opt: 227 buf = io.BytesIO() 228 with zipfile.ZipFile(buf, "w") as z: 229 dec = opt["daily_plan"][["day", "date", "status"] + data.var_names] 230 z.writestr("daily_decisions_optimized.csv", dec.to_csv(index=False)) 231 z.writestr("daily_plan.csv", opt["daily_plan"].to_csv(index=False)) 232 z.writestr("tank_balance.csv", opt["tank_balance"].to_csv(index=False)) 233 z.writestr("constraint_check.csv", opt["checks"].to_csv(index=False)) 234 z.writestr("binding_constraints.csv", res.binding.to_csv(index=False)) 235 downloads = buf.getvalue()	Results zip for the download button (decisions, daily plan, tank balance, checks, binding constraints) – only when a plan was found.
237 return {"model": clean(model), "result": clean(result), "summary": summary, 238 "status": "optimal" if res.success else "infeasible", "downloads": downloads}	Returns the bundle (already JSON-cleaned) with its status.
242 if "frames" not in st.session_state:	First visit only:
243 try: 244 st.session_state.source_url = st.secrets.get("DATA_URL", DEFAULT_SOURCE)	read DATA_URL from Streamlit secrets if set…
245 except Exception: # no secrets.toml 246 st.session_state.source_url = DEFAULT_SOURCE	…otherwise use the default URL;
247 try: 248 load_source(st.session_state.source_url)	load the data;
249 except Exception: 250 st.session_state.source_url = DEFAULT_SOURCE 251 load_source(DEFAULT_SOURCE)	if that fails, fall back to the default source;
252 st.session_state.last_nonce = None	no canvas action processed yet.
254 with st.sidebar:	Sidebar (hidden by default):
255 st.header("Data source")	heading;
256 url = st.text_input("GitHub folder or raw URL", st.session_state.source_url, 257 help="Folder containing the CSVs (directly or in a data/ subfolder).")	box to type a different GitHub folder;
258 if st.button("↻ Load from GitHub", use_container_width=True): 259 st.session_state.source_url = url 260 load_source(url) 261 st.rerun()	button reloads from that folder (and solves once) and restarts the script;
262 src = st.session_state.sources 263 n_gh = sum(v != "local" for v in src.values()) 264 st.caption(f"{n_gh}/{len(src)} files from GitHub, {len(src) - n_gh} from the bundled data/ folder.")	shows how many files came from GitHub vs local…

Line · statement(s)	What it does

265
266
`for k, v in src.items():`
`  st.caption(f"• {data_loader.FILES[k]} — {'local' if v == 'local' else 'GitHub'}")`
	…and the source of each file.

268 `frames = st.session_state.frames`	Working tables (include any applied edits).
269 `settings = st.session_state.settings`	Current settings.
270 `T = len(frames["daily_decisions"])`	Horizon length.
271 `settings["first_plan_day"] = int(min(max(settings["first_plan_day"], 1), T))`	Keeps the first plan day within 1…T.
274 `issues = data_loader.validate(frames)`	Validates the (possibly edited) tables on every script run – so Apply gives immediate error feedback even without solving.
275 `errors = [m for lvl, m in issues if lvl == "error"]`	Errors only.

276
277
`crudes = frames["crude_data"]["crude_id"].astype(str).tolist() if "crude_data" in frames else []`
`products = frames["product_data"]["product_id"].astype(str).tolist() if "product_data" in frames else []`
	Current crude and product IDs (for drop-downs).

279
280
281
282
283
284
285
286
287
288
289
290
`payload = {`
`  "ack": st.session_state.last_nonce,`
`  "source": {"url": st.session_state.source_url,`
`  "github_files": sum(v != "local" for v in st.session_state.sources.values()),`
`  "total_files": len(st.session_state.sources)},`
`  "tables": {n: table_payload(n, frames[n], crudes, products) for n in INPUT_TABLES},`
`  "edited": [n for n in INPUT_TABLES if not frames_equal(frames[n], st.session_state.base_frames[n])],`
`  "issues": [{"level": l, "msg": m} for l, m in issues],`
`  "settings": settings | {"T": T},`
`  "summary": {}, "model": None, "result": None,`
`  "status": "invalid" if errors else "pending", "stale": False,`
`}`
	Data sent to the canvas: ack (last processed action), data source, editable tables, edited tables, issues, settings, summaries, model, results, status, and **[NEW]** "stale" (True when outputs belong to an older run).

292 `bundle = None`	Will hold the results bundle to display.
293 `if errors:`	Inputs invalid:
294 `  payload["summary"] = {"validate": f"{len(errors)} error(s) — click to see"}`	only the validation block gets a summary;
295 `  st.session_state.run_requested = False  # nothing can be solved until the inputs are fixed`	**[NEW]** any pending Run is cancelled – nothing can be solved until the inputs are fixed.
296 `else:`	Inputs valid:
297 `  data = data_loader.build(frames, settings["first_plan_day"])`	build PlanData;

298
299
`  fingerprint = (tuple(int(pd.util.hash_pandas_object(f.astype(str), index=False).sum()) for f in frames.values()) ↵ ,`
`  settings["first_plan_day"], settings["soft"], float(settings["penalty"]))`
	fingerprint of every table + settings (the cache key);

300 `  cache = st.session_state.solve_cache`	results cache.

302
303
304
305
306
307
308
`  if st.session_state.run_requested or st.session_state.last_bundle is None:  # Run / settings / first load`
`  res = optimize(data, soft_tanks=settings["soft"], penalty_usd_bbl=float(settings["penalty"]))`
`  bundle = make_bundle(data, evaluate(data), res)`
`  if len(cache) > 8:`
`  cache.clear()`
`  cache[fingerprint] = bundle`
`  stale = False`
	[NEW] Decision 1 – SOLVE: only when Run optimizer / run settings / first load requested it (or nothing has ever been solved). Optimise, build the bundle, keep the cache small and store it. Outputs are up to date (stale = False).

Line · statement(s)	What it does
309 elif fingerprint in cache: # these exact inputs were solved before 310 bundle, stale = cache[fingerprint], False	**[NEW]** Decision 2 – REUSE: these exact inputs were solved earlier (e.g. after Reset or undoing an edit) → show those results instantly, no solve.
311 else: # inputs applied, not yet re-run 312 bundle, stale = st.session_state.last_bundle, True	**[NEW]** Decision 3 – KEEP LAST RUN: inputs were applied but not yet solved → keep showing the last run and mark it stale ("Re-run needed").
314 st.session_state.run_requested = False	**[NEW]** Clears the run request so the next Apply does not trigger a solve.
315 st.session_state.last_bundle = bundle	**[NEW]** Remembers what is being displayed as the latest results.
317 payload["model"], payload["result"] = bundle["model"], bundle["result"]	Puts the bundle's model and results into the payload…
318 payload["status"], payload["stale"] = bundle["status"], stale	…with its status and the stale flag.
320 # input-side summaries always reflect the CURRENT (applied) inputs	Comment: input summaries reflect the current, applied inputs.
321 cr, pr, tk, P = frames["crude_data"], frames["product_data"], frames["tank_master"], data.P	Short names for the summary lines.
322 payload["summary"] = { 323 "crude_data": f"{len(cr)} crudes · {cr.price_usd_bbl.min():g}–{cr.price_usd_bbl.max():g} $/bbl", 324 "unit_data": f"CDU {P['CDU_MAX']:g} · REF {P['REF_MAX']:g} · RFCC {P['RFCC_MAX']:g} · DHT {P['DHT_MAX']:g} k ↪ bd", 325 "product_data": f"{len(pr)} products · {pr.price_usd_bbl.min():g}–{pr.price_usd_bbl.max():g} $/bbl", 326 "tank_master": f"{len(tk)} tanks · {tk.opening_kbbl.sum():,.0f} kbbl opening", 327 "crude_receipts": f"{len(frames['crude_receipts'])} cargoes · {frames['crude_receipts'].qty_kbbl.sum():,.0f} ↪ kbbl", 328 "product_liftings": f"{len(frames['product_liftings'])} liftings · {frames['product_liftings'].qty_kbbl.sum(↪):,.0f} kbbl", 329 "daily_decisions": f"Actual 1–{data.first_plan_day - 1} · Plan {data.first_plan_day}–{data.T}", 330 "validate": "All checks passed" if not issues else f"{len(issues)} warning(s)", 331 }	One-line text for the input and Validate blocks (e.g. "9 cargoes · 4,000 kbbl") – updates as soon as you Apply.
332 # model / optimizer / output summaries come from the last run	Comment: output summaries come from the last run.
333 payload["summary"].update({k: bundle["summary"][k] for k in OUTPUT_SUMMARY_KEYS})	Copies the model/optimizer/output summaries from the bundle.
334 if stale: 335 payload["summary"]["optimizer"] = "Inputs changed — press Run"	**[NEW]** When stale, the optimizer block summary says "Inputs changed – press Run".
338 st.markdown("#### Refinery Production Planning — interactive optimizer flow")	Page title.
339 action = flow_canvas(payload=clean(payload), key="flow", default=None)	Draws the canvas with the cleaned payload. Returns the last user action sent from the browser.
341 if action and action.get("nonce") and action["nonce"] != st.session_state.last_nonce:	New action? (each action has a unique nonce so it is processed only once)
342 st.session_state.last_nonce = action["nonce"]	Remember it; the canvas uses "ack" to clear its busy state.
343 kind = action.get("action")	Action type:
344 if kind == "edit" and action.get("table") in INPUT_TABLES: # Apply: save only, no solve 345 frames[action["table"]] = rows_to_frame(action["table"], action["columns"], action["rows"], frames)	**[REVISED]** edit (the Apply button) → replace that table with the edited rows. No solve is requested.

Line · statement(s)	What it does
346 elif kind == "reset_table" and action.get("table") in INPUT_TABLES: 347 frames[action["table"]] = st.session_state.base_frames[action["table"]].copy()	reset_table → restore one table from the GitHub data (no solve; cached results are reused if available);
348 elif kind == "reset_all": 349 st.session_state.frames = {k: v.copy() for k, v in st.session_state.base_frames.items()} 350 settings["first_plan_day"] = default_first_plan_day(st.session_state.base_frames) 351 settings["soft"] = False	reset_all → restore all tables and default settings (no solve; cached results reused);
352 elif kind == "settings": # run settings: apply and solve 353 settings["first_plan_day"] = int(action.get("first_plan_day", settings["first_plan_day"])) 354 settings["soft"] = bool(action.get("soft", settings["soft"])) 355 settings["penalty"] = float(min(max(float(action.get("penalty", settings["penalty"])), 1.0), 1e6))	settings (Run settings → Apply & solve) → new first plan day, soft mode and penalty (clamped 1…1,000,000) …
356 st.session_state.run_requested = True	… and request a solve.
357 elif kind == "run": # Run optimizer: solve current inputs 358 st.session_state.run_requested = True	**[REVISED]** run (▶ Run optimizer) → request a solve of the current inputs. The cache is no longer cleared, so earlier results remain available for Reset/undo.
359 st.rerun()	Restart the script so the canvas shows the new state.
362 c1, c2, c3 = st.columns([1, 1, 2])	Three columns under the canvas:
363 with c1: 364 buf = io.BytesIO() 365 with zipfile.ZipFile(buf, "w") as z: 366 for n, df in st.session_state.frames.items(): 367 z.writestr(data_loader.FILES[n], df.to_csv(index=False)) 368 st.download_button("↓ Edited input CSVs", buf.getvalue(), "inputs_edited.zip", "application/zip", 369 use_container_width=True, help="Commit these to GitHub to make the scenario the new base case ↪ .")	Download button with the edited input CSVs (commit them to GitHub to keep a scenario);
370 with c2: 371 if bundle and bundle["downloads"] and not errors: 372 st.download_button("↓ Optimizer results" + (" (last run)" if payload["stale"] else ""), 373 bundle["downloads"], "optimizer_results.zip", "application/zip", 374 use_container_width=True)	**[REVISED]** Download button with the optimiser results of the displayed run; labelled "(last run)" when the inputs have changed since;
375 with c3: 376 st.caption("Click any block to open it. Input blocks are editable — **Apply** saves the change; " 377 "press **▶ Run optimizer** to re-solve. Sample data are illustrative only.")	**[REVISED]** Help text: Apply saves the change; ▶ Run optimizer re-solves.

6. run_optimizer.py – Command line

Same pipeline without a browser; writes CSV results. Used by the Excel VBA button. Unchanged in rev. 2.

Line · statement(s)	What it does
1 """Command-line runner (no browser): CSV in -> optimise -> CSV out. Also used by the Excel VBA button. 2 3 python run_optimizer.py # local ./data 4 python run_optimizer.py --url https://github.com/<user>/<repo>/tree/main/<folder> 5 python run_optimizer.py --first-plan-day 13 --soft 6 """	Runs the optimizer without a browser (also used by the Excel VBA button). Usage examples in the docstring. Unaffected by the Apply/Run change: the command line always solves.
7 import argparse 8 from pathlib import Path	argparse reads command-line options; Path handles folders.
10 import data_loader 11 from model import evaluate 12 from optimizer import explain, optimize	The three engine modules.
15 def main():	Main routine.
16 ap = argparse.ArgumentParser() 17 ap.add_argument("--data", default=str(data_loader.DATA_DIR), help="local folder with the CSVs") 18 ap.add_argument("--url", default=None, help="GitHub folder URL (overrides --data)") 19 ap.add_argument("--out", default="outputs") 20 ap.add_argument("--first-plan-day", type=int, default=None) 21 ap.add_argument("--soft", action="store_true", help="soft tank min-heel / end-target") 22 ap.add_argument("--penalty", type=float, default=1000.0)	Defines options: --data (local folder), --url (GitHub folder), --out (results folder), --first-plan-day, --soft, --penalty.
23 a = ap.parse_args()	Reads the options typed by the user.
25 frames = data_loader.read_csvs_from_url(a.url)[0] if a.url else data_loader.read_csvs(a.data)	Loads from GitHub if --url is given, otherwise from the local folder.
26 errors = [m for lvl, m in data_loader.validate(frames) if lvl == "error"] 27 if errors: 28 raise SystemExit("Input errors:\n- " + "\n- ".join(errors))	Validates; any error stops the run with the list of problems.
29 d = data_loader.build(frames, a.first_plan_day)	Builds PlanData.
30 base = evaluate(d)	Evaluates the current plan.
31 res = optimize(d, soft_tanks=a.soft, penalty_usd_bbl=a.penalty)	Solves the LP.
32 print(f"Horizon {d.T} days | locked 1-{d.first_plan_day - 1} | solver: {res.message}") 33 if not res.success: 34 raise SystemExit(1)	Prints horizon and solver status; stops if not solved.
35 opt = evaluate(d, res.V)	Evaluates the optimised plan.
36 out = Path(a.out) 37 out.mkdir(exist_ok=True)	Creates the output folder.
38 opt["daily_plan"][["day", "date", "status"] + d.var_names].to_csv(out / "daily_decisions_optimized.csv", index=F ↪ alse) 39 opt["daily_plan"].to_csv(out / "daily_plan.csv", index=False) 40 opt["tank_balance"].to_csv(out / "tank_balance.csv", index=False) 41 opt["checks"].to_csv(out / "constraint_check.csv", index=False) 42 res.binding.to_csv(out / "binding_constraints.csv", index=False)	Writes optimised decisions, daily plan, tank balance, constraint check and binding constraints as CSVs.

Line · statement(s)	What it does
43 print(f"Current plan margin : {base['kpi']['plan_margin_kusd']:,.2f} $k") 44 print(f"Optimised margin : {opt['kpi']['plan_margin_kusd']:,.2f} $k") 45 print(f"Violations : {opt['kpi']['violations']}")	Prints the key numbers...
46 for line in explain(d, base["kpi"], opt["kpi"], res): 47 print(" -", line.replace("**", ""))	...the explanation sentences...
48 print(f"Results -> {out.resolve()}")	...and where the files were saved.
51 if __name__ == "__main__": 52 main()	Runs main() only when the file is started directly.

7. flow_component/index.html – interactive canvas

One self-contained HTML file (HTML + CSS + plain JavaScript, no external libraries). Streamlit loads it in an iframe. It never calculates plant results itself: it draws what Python sends and sends user actions back. Rows changed in rev. 2 are shaded.

Part	Key statements	What it does
CSS – colours	:root { --input, --calc, --core, --output, --bad, --warn … }	Colour palette: inputs blue, calculation purple, optimizer orange, outputs green, errors red, warnings amber.
CSS – layout	.toolbar, .chip, .btn, #stage, .node, .port, .colhead	Top bar with status chips and buttons; a 1400 × 730 px canvas with a dotted grid; block cards with ports and a hover glow per block type.
CSS – rev. 2 states	.btn.apply, .pulse, .node.stale, .core-z.old, .pill.warn, .stalebar	**[NEW]** Blue Apply button; pulsing animation for Run buttons; dashed amber border for out-of-date blocks; greyed Z value; amber RE-RUN NEEDED pill; amber banner inside output pop-ups.
CSS – optimizer block	.core-eq, .core-z, .pill, .runbtn, .busy-ring	Objective formula, Z value, status pill, Run button and the rotating ring shown while solving.
CSS – pop-ups	#modal, .panel, table, td input, td.changed, td.badcell, .kpi, .card, .eq, .tabs	Pop-up window, editable table cells (changed = yellow, invalid number = red), KPI cards, formula boxes and tabs.
CSS – accessibility	@media (prefers-reduced-motion)	Stops spinner, pulse and particle motion for users who turn off animations.
HTML body	#toolbar, #viewport › #stage › svg#edges, canvas#particles, #nodes, .hint, #modal	**[REVISED]** Containers: toolbar, canvas stage with SVG connection lines, particle layer, block layer and pop-up layer. The hint now reads 'edit an input and press Apply · then ▶ Run optimizer to re-solve'.
Streamlit bridge	inStreamlit; send(type,data); setValue(v); setHeight(h)	Talks to Streamlit with window.postMessage: setValue sends a user action to Python, setHeight resizes the iframe.
Block layout	W, H; COL; ICON + svgIcon(); NODES[… 15 blocks …]; byId; EDGES	Defines the 15 blocks (id, type, title, icon, CSV name, position) and the connections: every input → Validate → Model → Optimizer → every output.
State & helpers	P, busyNonce, busyKind, openId, draft, modalTab, hoverId; isStale(), solving(); $, esc, fmt, sgn, md	**[REVISED]** P = latest payload; busyNonce = action waiting for a reply; busyKind = which action it is (new). isStale() is true when P.stale (outputs belong to an older run); solving() is true only while a run or settings action is pending, so Apply never shows 'SOLVING…'.
act(obj), runOptimizer()	busyNonce = nonce; busyKind = obj.action; setValue({nonce, …obj}); renderBusy()	**[REVISED]** Sends one action (edit, reset_table, reset_all, settings, run) with a unique ID and remembers its kind. runOptimizer() is the shared handler for every ▶ Run button. A 30 s safety timer clears the busy state.
fit()	scale = min(width/1400, 1.25); transform; setHeight(…)	Scales the canvas to the page width and tells Streamlit how tall the iframe must be (taller while a pop-up is open).
renderToolbar()	status, chips, #tbReset, #tbRun	**[REVISED]** Status is Optimal / Infeasible / Input errors / Re-run needed. When stale, adds the chip 'Inputs changed – press ▶ Run optimizer', labels margins '(last run)' and pulses the Run button. Run is disabled while busy or when inputs have errors.

Part	Key statements	What it does
nodeState(n), matchIssue()	err / edited / warn / stale / dim / ok	**[REVISED]** An input is 'err' if a validation error names it, 'edited' if it differs from GitHub. When stale, Process model, LP optimizer and all outputs are 'stale' (amber dot, dashed border); otherwise outputs are 'err' when there is no solution; all dimmed when inputs are invalid.
sparkline()	<polyline> per tank, PLAN line	**[REVISED]** Mini chart of all tank fill levels inside the Tank level curves block; drawn at half opacity when stale.
optimizerPill()	SOLVING… / INPUT ERROR / RE-RUN NEEDED / OPTIMAL / INFEASIBLE	**[NEW]** Chooses the optimizer block's status pill in that priority order.
renderNodes()	column headings; div.node per block; data-run → runOptimizer()	**[REVISED]** Builds every block: ports, status dot, EDITED badge, icon, title, CSV name and summary. Outputs heading shows '· last run' when stale. The optimizer block shows the objective, Z (greyed when stale), uplift, pill and a pulsing ▶ Run button.
renderBusy()	toggles .busy; pill.outerHTML = optimizerPill()	**[REVISED]** Updates toolbar, spinner and pill without redrawing everything; the spinner only appears while solving.
edgeColor(), edgeActive()	red if error; amber from optimizer when stale; inactive downstream	**[REVISED]** Colours each connection. When stale, the optimizer → output connections turn amber, dashed and stop flowing, showing that outputs wait for the next Run.
renderEdges()	path d='M… C…' (cubic Bézier); linearGradient; getPointAtLength() × 81; parts[]	Draws a curved connection between ports with a colour gradient, samples 81 points along it, and creates particles (more for longer lines).
highlightEdges(), sizeCanvas()	strokeOpacity / strokeWidth; devicePixelRatio × scale	Brightens the hovered block's connections; keeps particles sharp on high-resolution screens.
tick(now) – animation	p.t += dt × speed; interpolate(pts); radial gradient; requestAnimationFrame(tick)	**[REVISED]** Animation loop (~60 fps): moves particles along active curves and draws glowing dots. Particles speed up ×2.8 only while solving (not on Apply).
Charts	scaleLin, niceTicks, lineChart(o), barChart(o), legend(), CRUDE_COLORS	Small SVG chart builders: axes, PLAN shading after the first plan day, limit lines, target triangles, grouped or stacked bars with tooltips.
Pop-up framework	openModal(id), closeModal(), frame(n, body, foot), #bnRun, renderModal(), needResult()	**[REVISED]** Opens/closes the pop-up and draws header, body and footer. New: when stale, frame() adds an amber banner 'these are the last-run results' with its own ▶ Run optimizer button to Process model, LP optimizer and every output pop-up.
modalTable(n) – input editor	draft; <select>/<input> per cell; changedCount(); footMsg(); #mAdd; #mReset; #mApply = 'Apply'	**[REVISED]** Editable table with highlighted changed/invalid cells, Enter to move down, add/delete rows for cargo tables. The button is now **Apply** (was 'Apply & re-optimize'): it sends {action:'edit'} which only saves the table. Footer shows 'N unsaved change(s) – press Apply', then 'Applied · press ▶ Run optimizer to re-solve'.
modalValidate(n)	issues list; checks card	**[REVISED]** Shows errors/warnings and the checks performed; text now explains validation runs on Apply and the optimizer on ▶ Run.
modalModel(n)	product equations with live yields; margin-per-barrel table	Explains how products are made from x, r and f, and which decision earns the most per barrel.
modalOptimizer(n)	tabs: objective / constraints / settings; #fp, #soft, #pen; #mApplyS; #mRun	**[REVISED]** Objective, constraints C1–C16 (labelled 'parameters of the last run') and Run settings. Settings button is still 'Apply & solve' (settings change the problem, so they solve). Footer says 'Inputs changed – press Run' and pulses ▶ Run optimizer when stale.

Part	Key statements	What it does
modalDecisions(n)	stacked crude-slate barChart; table with (change) vs current plan	Daily x, r, f with CDU, RON, sulfur and margin, and the change against the current plan.
modalEconomics(n)	KPI cards; daily margin bars; economics and product-make tables	Plan-days revenue, crude cost, opex and margin (current vs optimised) and product volumes.
modalTankBalance(n)	#tkSel; opening/inflow/outflow/closing/fill/status; C14 table	Day-by-day balance of the selected tank and the end-of-horizon check for all tanks.
modalTankCurves(n)	lineChart per tank; filter All / Crude / Product	Closing inventory curves (current dashed, optimised solid) vs capacity, min heel and end target.
modalWhy(n)	explain sentences; shortfall table; binding table	Plain-language reasons, tank shortfalls in soft mode, and binding constraints with shadow prices.
renderAll()	renderToolbar(); renderNodes(); renderEdges(); renderModal(); fit()	Full redraw after new data, keeping the scroll position of an open pop-up.
Message listener	'streamlit:render' → P = args.payload; if P.ack === busyNonce → clear busyNonce, busyKind	**[REVISED]** Receives each payload from Python, clears the busy state (and its kind) when Python confirms the action, and redraws.
Start-up	componentReady; renderAll(); requestAnimationFrame(tick); window.__flow	Tells Streamlit the component is ready, draws the first frame, starts the animation, and exposes a small hook used for automated testing.

8. Appendix – input files and statement coverage

CSV file	Used by	Content
crude_data.csv	coeffs(), validate()	Price, CDU cut yields, residue sulfur, min/max charge per crude
unit_data.csv	coeffs(), optimize()	param_key / value: capacities, ramp, opex, unit yields, RON and sulfur specs
product_data.csv	coeffs()	Product IDs, tanks and netback prices
tank_master.csv	tank_vector(), optimize()	Capacity, min heel, opening stock, end-of-horizon target
crude_receipts.csv	build() → R	Cargo arrivals: day, crude, kbbl
product_liftings.csv	build() → L	Liftings: day, product, kbbl
daily_decisions.csv	build() → V	Recorded / current plan decisions per day and Actual/Plan status
config.csv	build() → cfg	Settings such as first_plan_day and the benchmark margin

Python file	Lines	Explained statement groups	Changed in rev. 2
data_loader.py	269	108	–
model.py	163	107	–
optimizer.py	181	107	–
app.py	377	127	17
run_optimizer.py	52	19	–